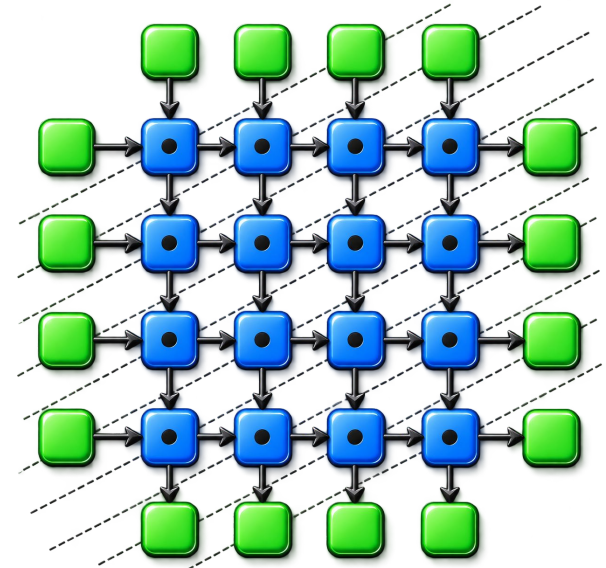
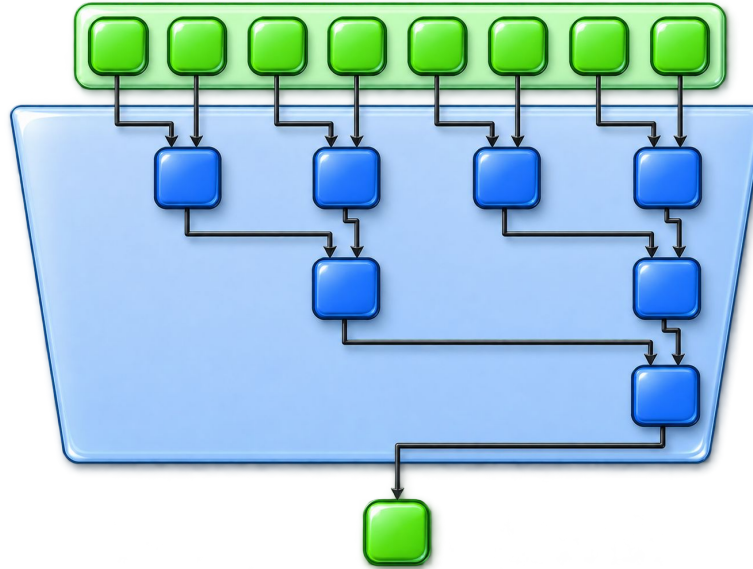
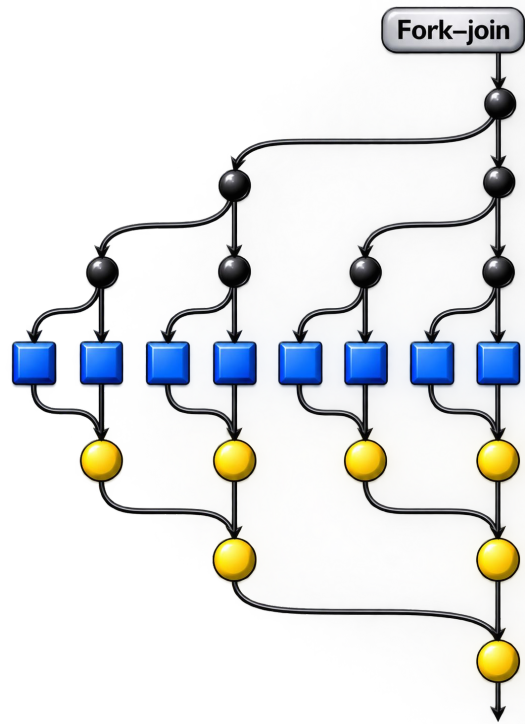




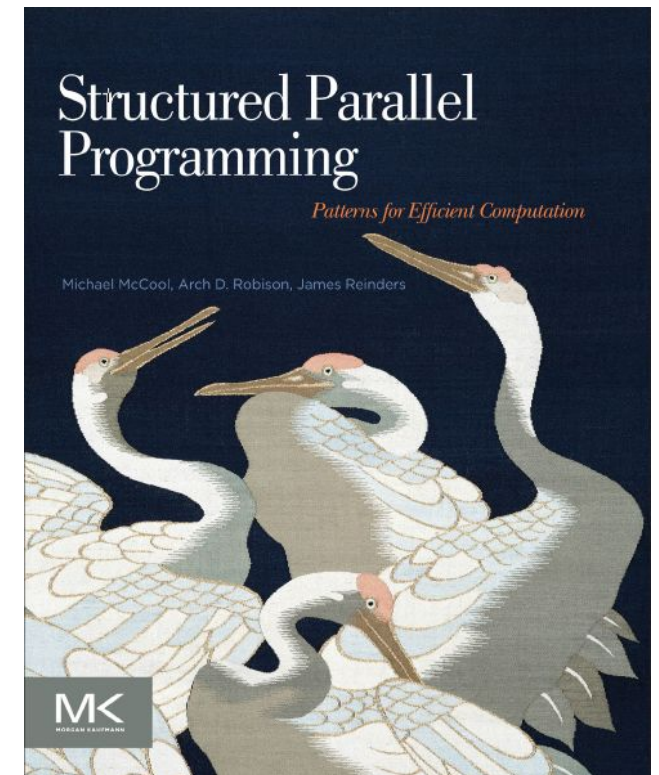
Patrones paralelos

Septiembre/2026

PATRONES PARALELOS

1. Introducción
2. Nesting Pattern
3. Patrones de control seriales
4. Patrones de control paralelos
5. Patrones seriales de gestión de datos
6. Patrones de gestión de datos paralelos
7. Otros patrones paralelos
8. Soporte de modelos de programación para patrones
9. Tres libros más...
10. Para terminar: compilando para paralelismo

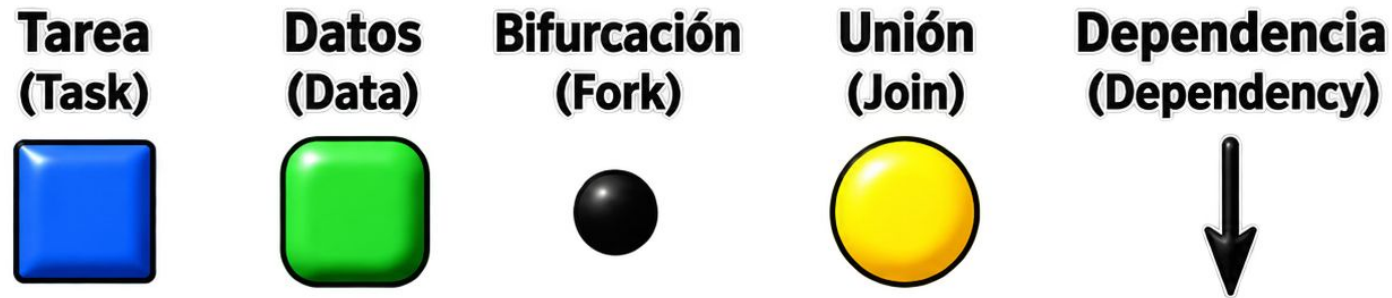
Vamos a dar una
mirada rápida al
capítulo 3 este libro



INTRODUCCIÓN

- ❑ Patrones paralelos: una combinación recurrente de distribución de tareas y acceso a datos que resuelve un problema específico en el diseño de algoritmos paralelos.
- ❑ Los patrones nos proporcionan un "vocabulario" para el diseño de algoritmos
- ❑ Puede ser útil comparar patrones paralelos con patrones seriales.
- ❑ Los patrones son universales: se pueden utilizar en cualquier sistema de programación en paralelo

NOTACIÓN GRÁFICA



Nuestra notación gráfica para los componentes fundamentales de los algoritmos: tareas y datos. Usamos dos símbolos adicionales para representar la bifurcación y la fusión de flujos de control seriales mediante fork y join, y flechas para representar dependencias.

TAREAS

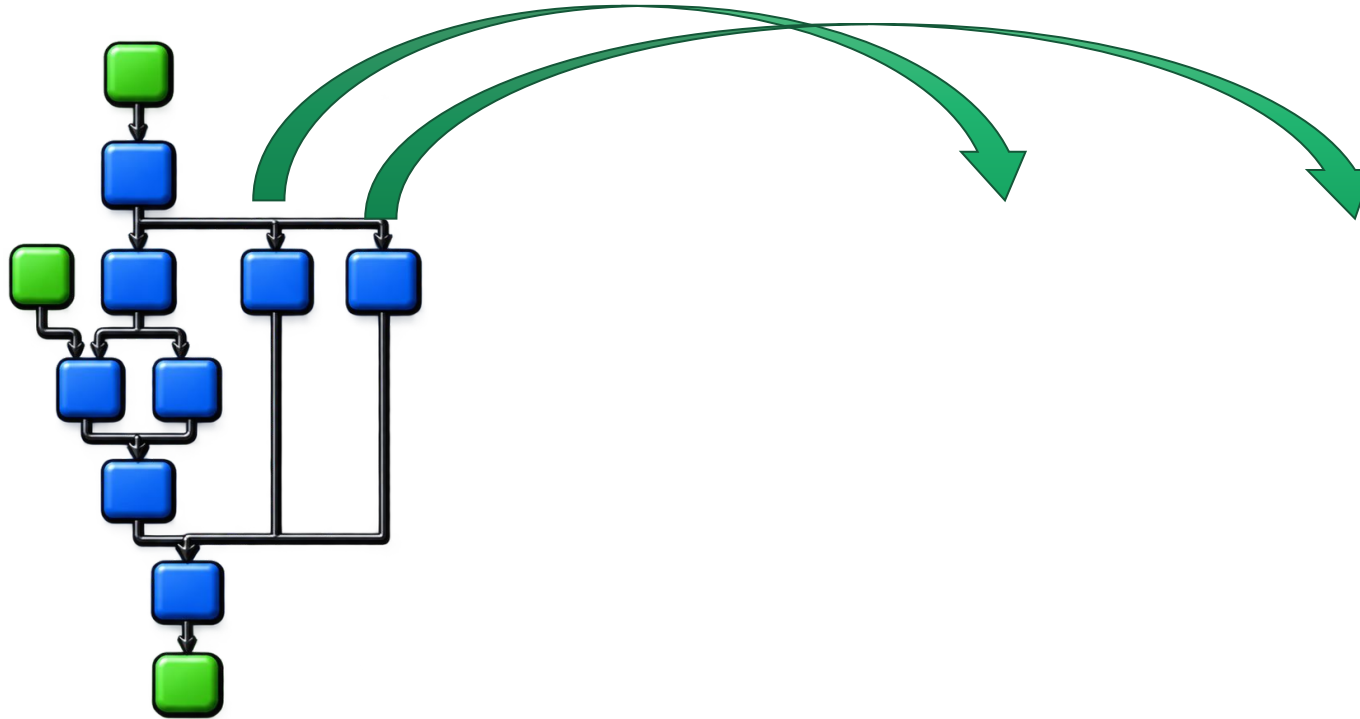
- ❑ Para el capítulo 1 deben contestar un formulario de Google de manera individual.
- ❑ Para el capítulo 2 deben responder 5 preguntas de un cuestionario y subir las respuestas al classroom, también de forma individual.



NESTING PATTERN

- ❑ Nesting es la capacidad de componer [acomodar] patrones jerárquicamente
- ❑ Este patrón aparece en ambos tipos de algoritmos: **seriales** y **paralelos**
- ❑ Diagramas de patrones [“Pattern diagrams”] se utilizan para mostrar visualmente la idea del patrón donde cada "bloque de tareas" es una ubicación de código general en un algoritmo
- ❑ Cada "bloque de tareas" puede ser a su vez otro patrón en el patrón de anidamiento [nesting pattern]

PATRÓN DE ANIDAMIENTO [NESTING PATTERN]



Nesting Pattern: El anidamiento permite “acomodar” otros patrones en una jerarquía para que cualquier bloque de tareas en el diagrama anterior se pueda reemplazar con un patrón con la misma entrada/salida y las mismas dependencias.

PATRONES DE CONTROL SERIALES

- La programación estructurada “serial” se basa en estos patrones:
 1. sequence,
 2. selection,
 3. iteration, y
 4. recursion
- El patrón de anidamiento también se puede utilizar para componer jerárquicamente estos cuatro patrones.
- Aunque usted debe conocer ya estos patrones, es importante comprender estos patrones al paralelizar algoritmos seriales basados en estos patrones.

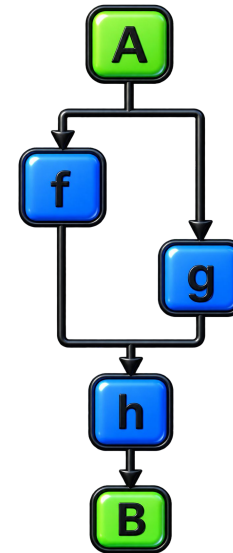
PATRONES DE CONTROL SERIALES: SEQUENCE

- ❑ Secuencia: lista ordenada de tareas que se ejecutan en un orden específico
- ❑ Supuesto: se sigue el orden del texto [instrucciones] del programa (es obvio, pero esto será importante cuando se paralelice)

```
1  T = f(A);  
2  S = g(T);  
3  B = h(S);
```



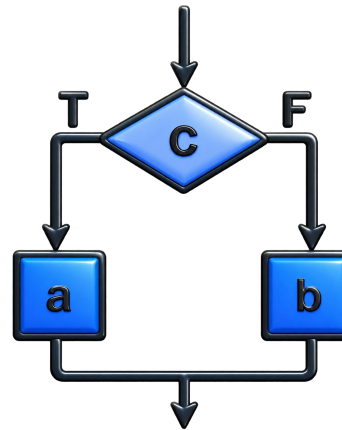
```
1  T = f(A);  
2  S = g(A);  
3  B = h(S,T);
```



PATRONES DE CONTROL SERIALES: SELECTION

- ❑ Selection: primero se evalúa la condición c . La tarea a o la tarea b es ejecutada dependiendo de si el resultado de c es verdadero (true) o falso (false).
- ❑ Supuestos – a y b nunca son ejecutados antes que c , y solo a o b es ejecutado – nunca ambos.

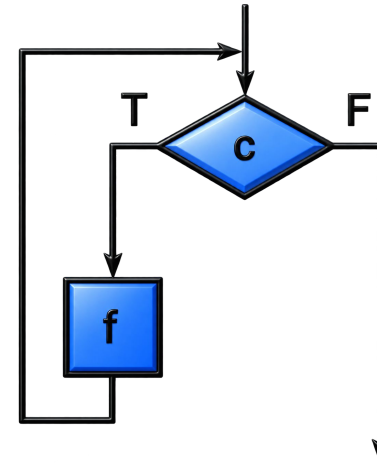
```
1  if (c) {  
2    a;  
3  } else {  
4    b;  
5  }
```



PATRONES DE CONTROL SERIALES: ITERATION

- ❑ Iteración: una condición c es evaluada. Si es cierta, a es evaluada, y luego c se evalúa de nuevo. Esto se repite hasta que c sea falso.
- ❑ Complicación al paralelizar: posibilidad de que existan dependencias con iteraciones anteriores

```
1  for (i = 0; i < n;  
2    a;  
3  }  
-----  
1  while (c) {  
2    a;  
3  }
```



PATRONES DE CONTROL SERIALES: RECURSION

- ❑ Recursión: es *una forma dinámica de anidamiento* que permite a las funciones llamarse a sí mismas (asociado con la asignación de memoria basada en una pila –LIFO–)

Leer el documento ¿Qué es la recursividad?

(O lo que es lo mismo, leer el capítulo 1 del libro de “THE RECURSIVE BOOK OF RECURSION” de Al Sweigart. 2022. No Starch Press.

Ver el video !!Con 2019- Tail Call Optimization: The Musical!! by Anjana Vakil & Natalia Margolis

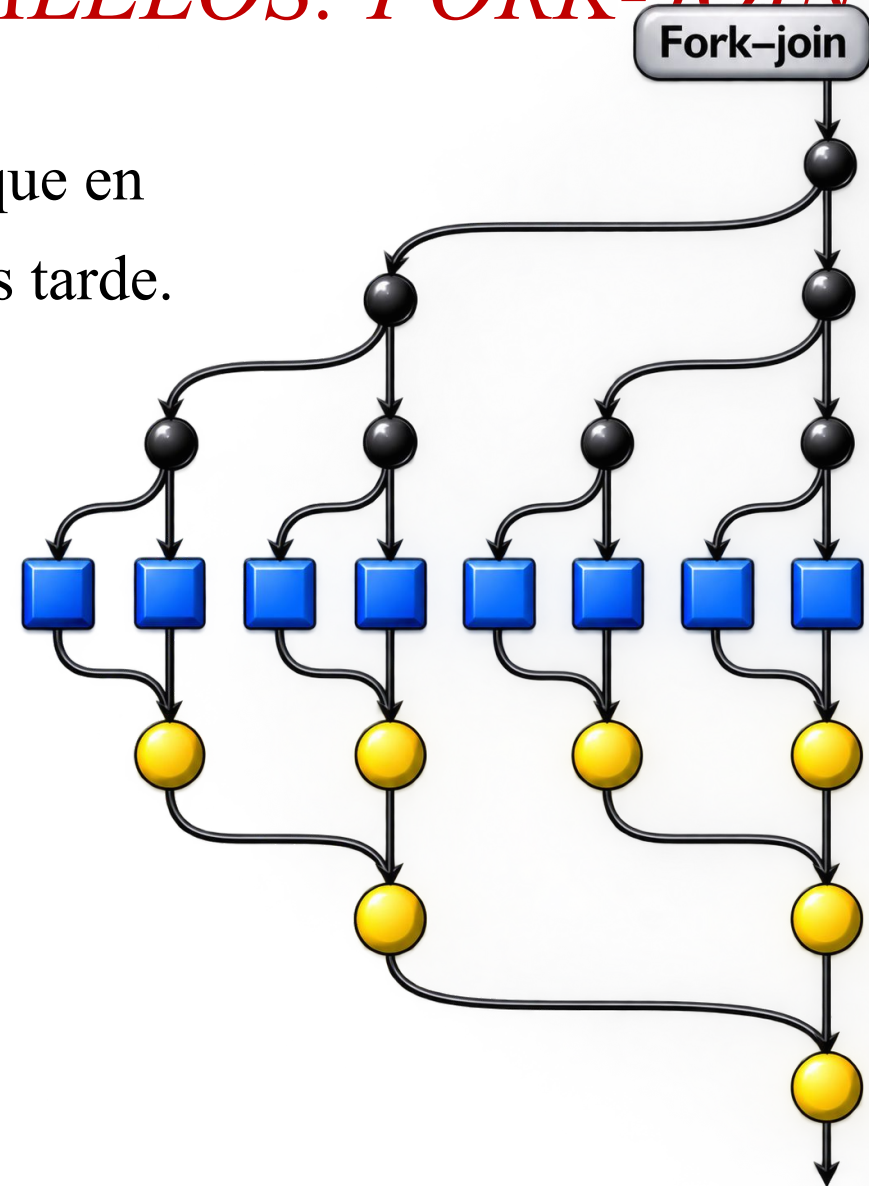
(<https://www.youtube.com/watch?v=-PX0BV9hGZY>)

PATRONES DE CONTROL PARALELOS

- Los patrones de control paralelos extienden los patrones de control en serie
- Cada patrón de control paralelo está relacionado con al menos un patrón de control en serie, pero suaviza los supuestos de los patrones de control en serie.
- Patrones de control paralelos
 1. fork-join,
 2. map,
 3. stencil,
 4. reduction,
 5. scan,
 6. recurrence

PATRONES DE CONTROL PARALELOS: FORK-JOIN

- Fork-join: Permite que el flujo de control se bifurque en múltiples flujos paralelos, luego se reincorpore más tarde.

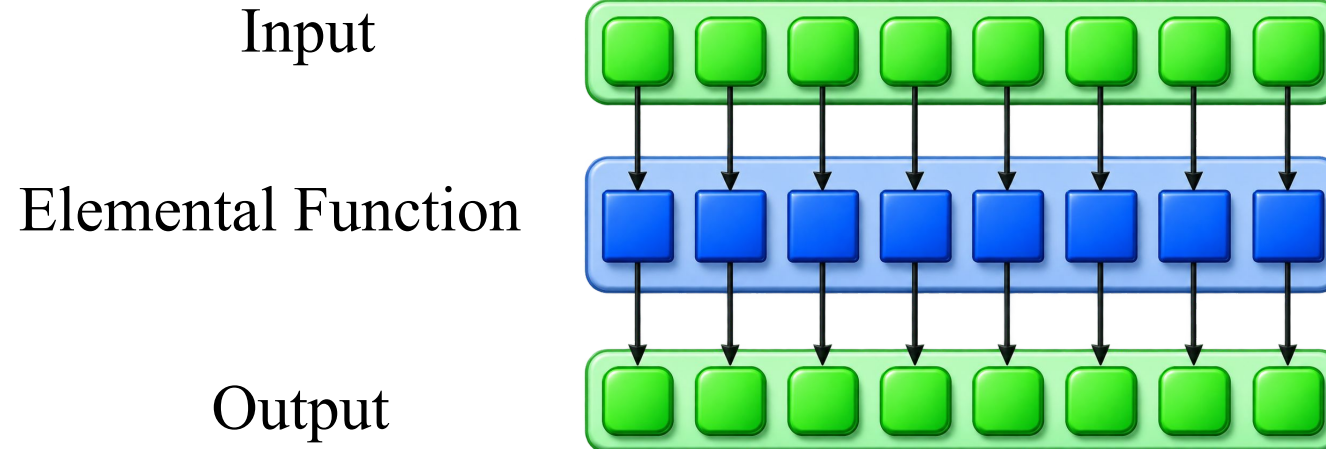


PATRONES DE CONTROL PARALELOS: FORK-JOIN

- ❑ Fork-join: Permite que el flujo de control se bifurque en múltiples flujos paralelos, luego se reincorpore más tarde
- ❑ Cilk Plus lo implementa con `spawn` y `sync`
 - El árbol de llamadas es un árbol de llamadas paralelo y las funciones se generan `[spawn]` en lugar de llamarse
 - Las funciones que generan `[spawn]` otra llamada de función continuarán ejecutándose
 - Invocando *syncs* con la función generada `[spawn]` para unir las dos
- ❑ Un “join” es diferente a una barrera [“barrier”]
 - Sync – solo un hilo continua
 - Barrier – todos los hilos continúan

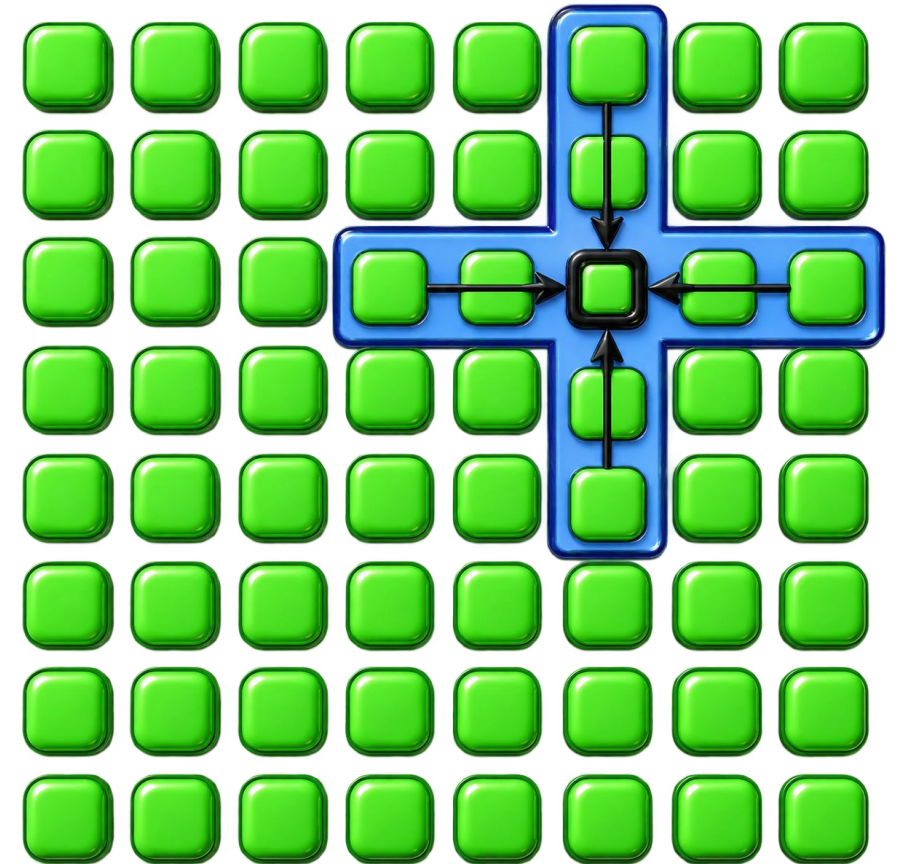
PATRONES DE CONTROL PARALELOS: MAP

- ❑ Map: realiza una función sobre cada elemento de una colección
- ❑ Map replica un patrón de iteración en serie donde cada iteración es independiente de las demás, el número de iteraciones se conoce de antemano y el cálculo solo depende del recuento de iteraciones y los datos de la colección de entrada
- ❑ La función replicada se conoce como una "función elemental"



PATRONES DE CONTROL PARALELOS: STENCIL

- Stencil: Una función elemental accede a un conjunto de "vecinos", la plantilla es una generalización del patrón map
- A menudo combinado con iteración
 - utilizado con solucionadores
 - iterativos o para “evolucionar”
 - un sistema a través del tiempo
- Las condiciones de contorno [frontera] deben manejarse con cuidado en el patrón de stencil

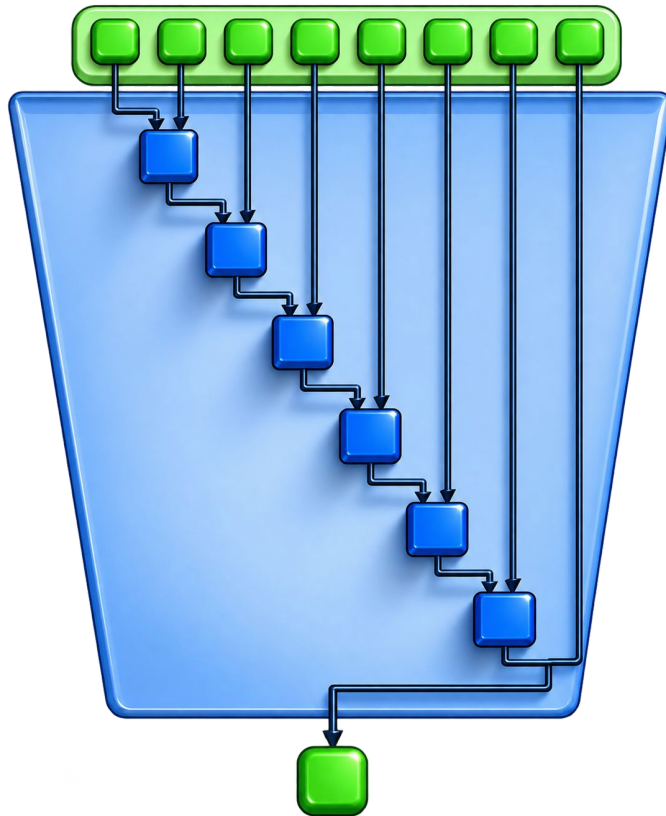


PATRONES DE CONTROL PARALELOS: REDUCTION

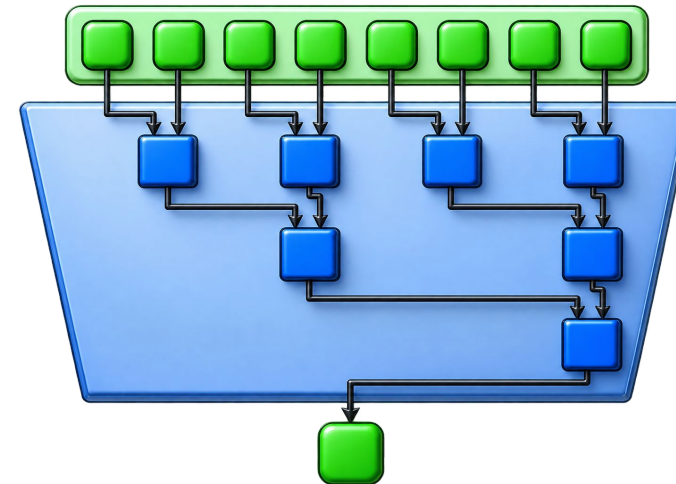
- ❑ Reduction:
- ❑ Combina todos los elementos de una colección mediante una "función de combinación" *asociativa*
- ❑ Debido a la asociatividad de la función que combina, es posible usar diferentes ordenamientos de la reducción
- ❑ Ejemplos de “funciones de combinación”: suma, multiplicación, máximo, mínimo y los operadores booleanos AND, OR y XOR

PATRONES DE CONTROL PARALELOS: REDUCTION

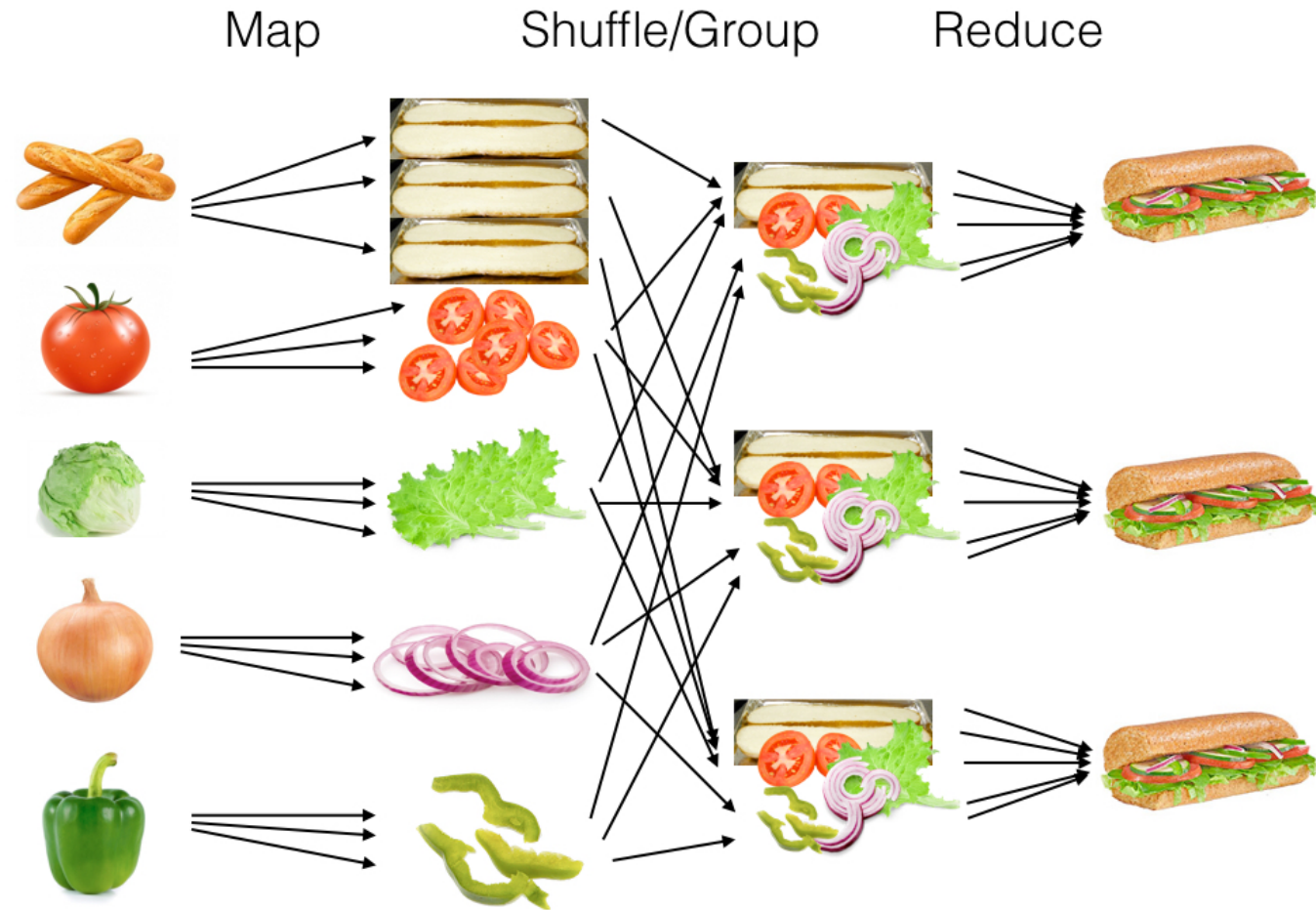
Serial Reduction



Parallel Reduction



PATRONES DE CONTROL PARALELOS: REDUCTION



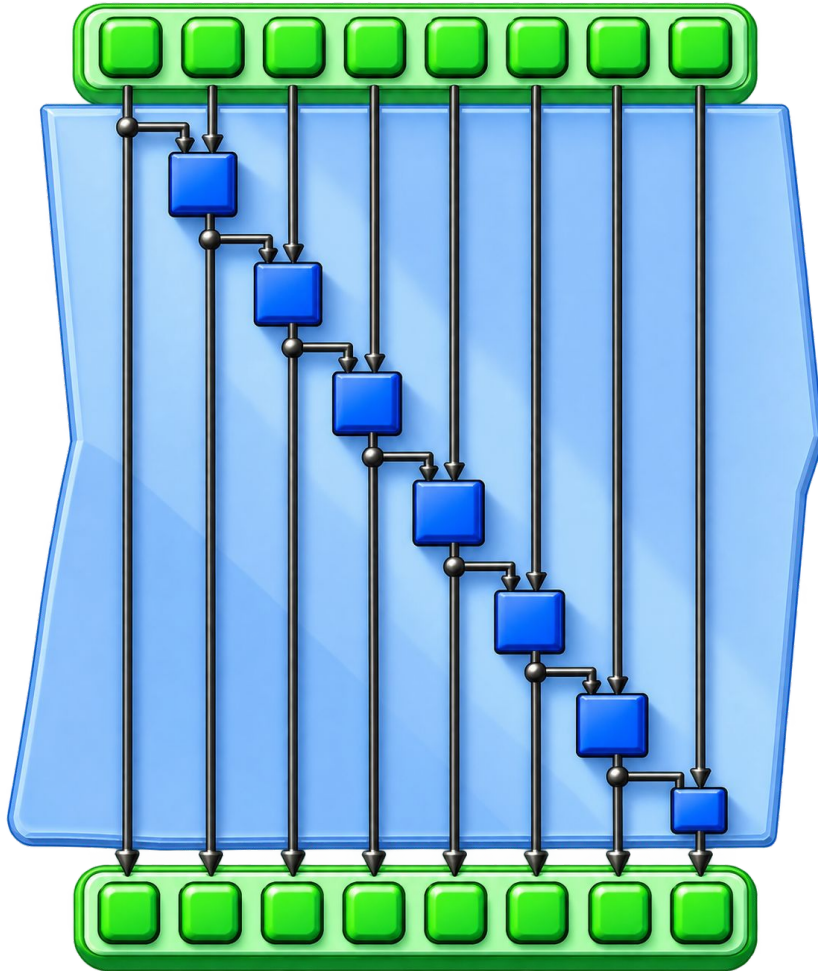
Patrón
MAP-REDUCE

PATRONES DE CONTROL PARALELOS: SCAN

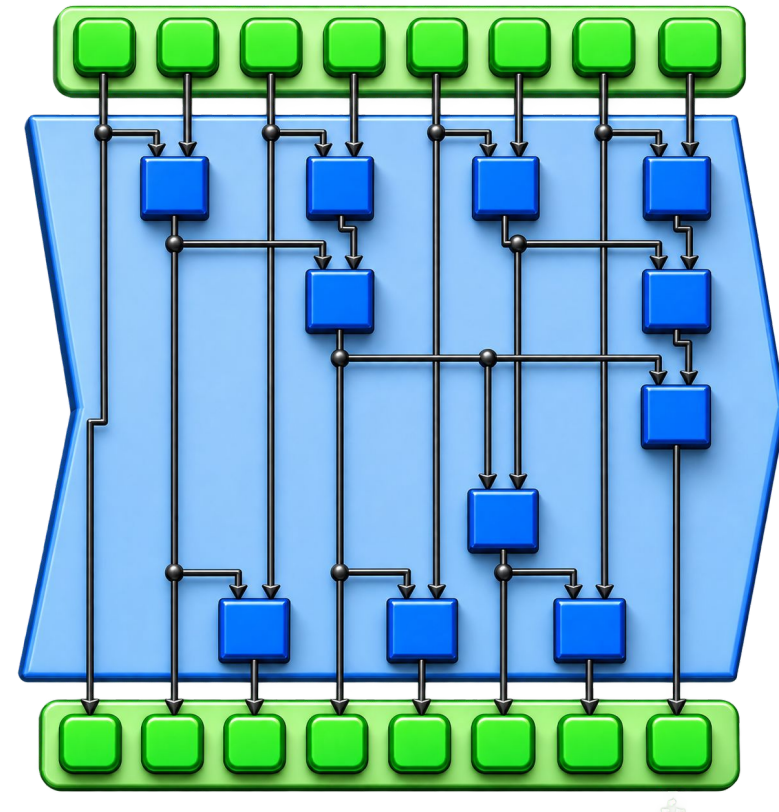
- ❑ Scan: calcula toda la reducción parcial de una colección
- ❑ Para cada salida de una colección, se calcula una reducción de la entrada hasta ese punto.
- ❑ Si la función que se utiliza es asociativa, el scan se puede paralelizar
- ❑ La puesta en paralelo de un scan no es obvia al principio, debido a las dependencias de iteraciones anteriores en el ciclo [loop] en serie
- ❑ Un scan paralelo requerirá más operaciones que una versión en serie

PATRONES DE CONTROL PARALELOS: SCAN

Serial Scan

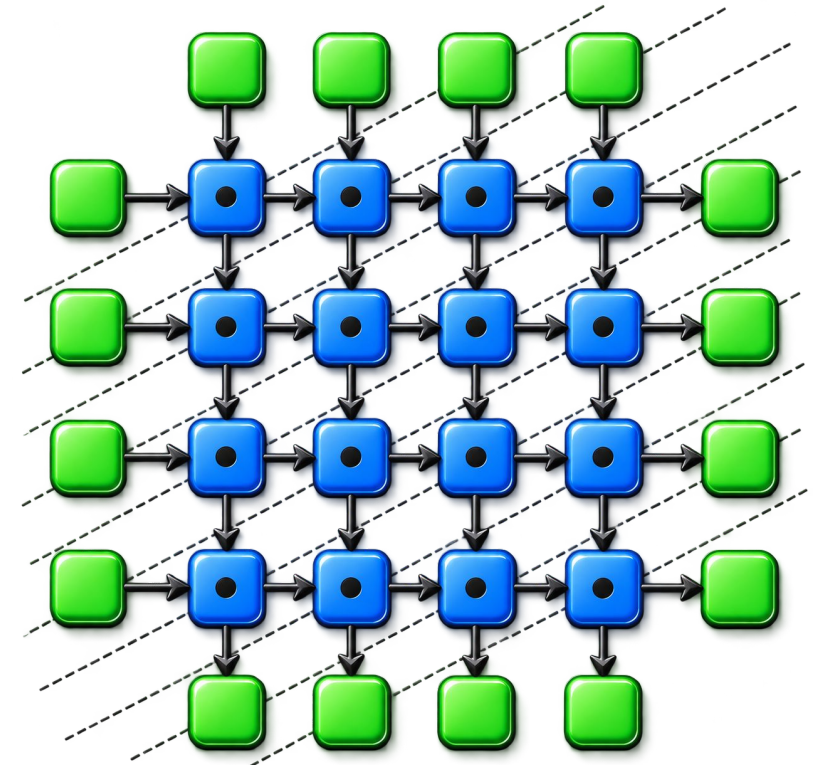


Parallel Scan



PATRONES DE CONTROL PARALELOS: RECURRENCE

- ❑ Recurrencia: versión más compleja de map, donde las iteraciones de ciclo [loop] pueden depender unas de otras
- ❑ Similar a map, pero los elementos pueden usar salidas de elementos adyacentes como entradas
- ❑ Para que una recurrencia sea computable, debe haber un ordenamiento serial de los elementos de recurrencia de tal modo que los elementos se puedan computar usando salidas [resultados] previamente calculadas



Leer

Patrones_de_control_paralelo.pdf

PATRONES SERIALES DE GESTIÓN DE DATOS

- Los programas seriales pueden administrar datos de muchas maneras
- La gestión de datos se ocupa de cómo se asignan, comparten, leen, escriben y copian los datos
- Patrones seriales de gestión de datos:
 1. random read y random write,
 2. stack allocation,
 3. heap allocation,
 4. objects

PATRONES SERIALES DE GESTIÓN DE DATOS: RANDOM READ Y RANDOM WRITE

- ❑ Ubicaciones de memoria indexadas con direcciones
- ❑ Los punteros se utilizan normalmente para referirse a direcciones de memoria.
- ❑ El aliasing (incertidumbre de dos punteros que se refieren al mismo objeto) puede causar problemas cuando el código serial está en paralelo

PATRONES SERIALES DE GESTIÓN DE DATOS: STACK ALLOCATION

- ❑ La asignación de pila es útil para asignar datos dinámicamente en forma LIFO
- ❑ Eficiente: se puede asignar una cantidad arbitraria de datos en tiempo constante
- ❑ La asignación de pilas también preserva la localidad
- ❑ Cuando se paraleliza, normalmente cada hilo obtendrá su propia pila, por lo que se conserva la localidad del hilo

PATRONES SERIALES DE GESTIÓN DE DATOS: HEAP ALLOCATION

- ❑ La asignación Heap es útil cuando los datos no se pueden asignar de forma LIFO.
- ❑ Pero, la asignación Heap es más lento y más complejo que la asignación de pila
- ❑ Se debe utilizar una Asignación heap paralelizada al asignar memoria dinámicamente en paralelo
- ❑ Este tipo de asignador mantendrá grupos separados para cada trabajador [hilo] paralelo

PATRONES SERIALES DE GESTIÓN DE DATOS: OBJETOS

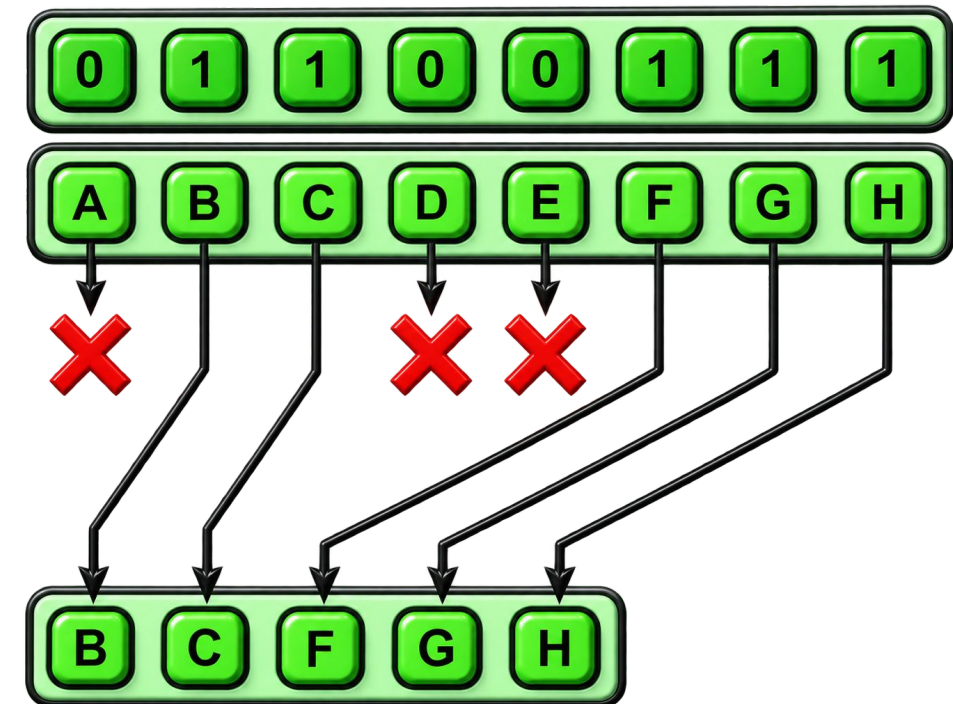
- ❑ Los objetos son construcciones de lenguaje para asociar datos con código para manipular y administrar esos datos.
- ❑ Los objetos pueden tener funciones miembro y también se consideran miembros de una clase de objetos.
- ❑ Los modelos de programación paralela generalizarán los objetos de varias formas

PATRONES DE GESTIÓN DE DATOS PARALELOS

- Para evitar situaciones como las condiciones de carrera, es de suma importancia saber cuándo varios trabajadores paralelos pueden compartir datos y cuándo no.
- Algunos patrones de gestión de datos paralelos nos ayudan con la localidad de los datos
- Patrones de gestión de datos paralelos:
 1. pack,
 2. pipeline,
 3. geometric decomposition,
 4. gather, y
 5. scatter

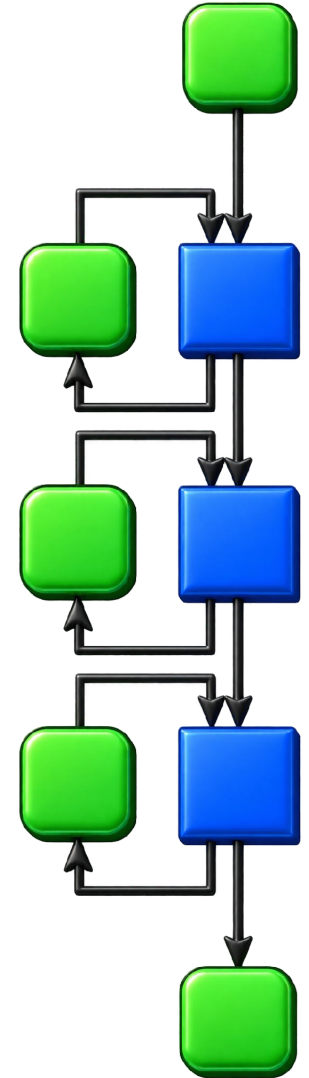
PATRONES DE GESTIÓN DE DATOS PARALELOS: PACK

- ❑ Pack se utiliza para eliminar el espacio no utilizado en una colección
- ❑ Los elementos marcados con false se descartan, los elementos restantes se colocan en una secuencia contigua *en el mismo orden*
- ❑ Útil cuando se utiliza con el patron map
- ❑ Unpack es el patrón inverso y se utiliza para volver a colocar elementos en sus ubicaciones originales



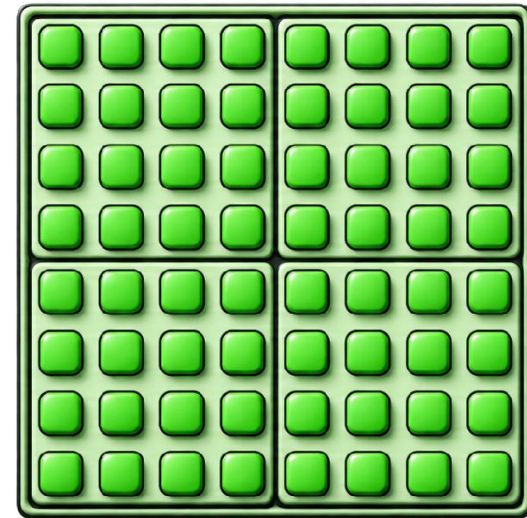
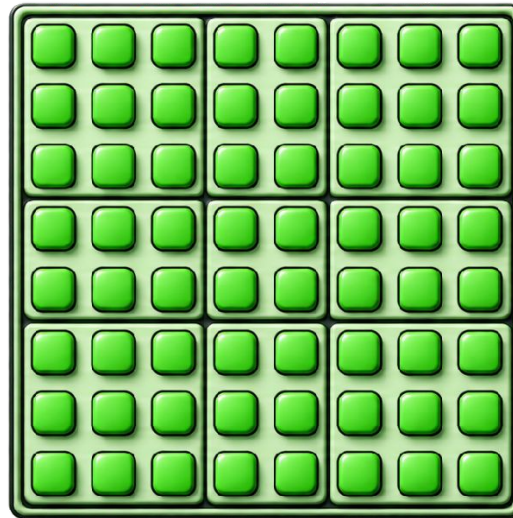
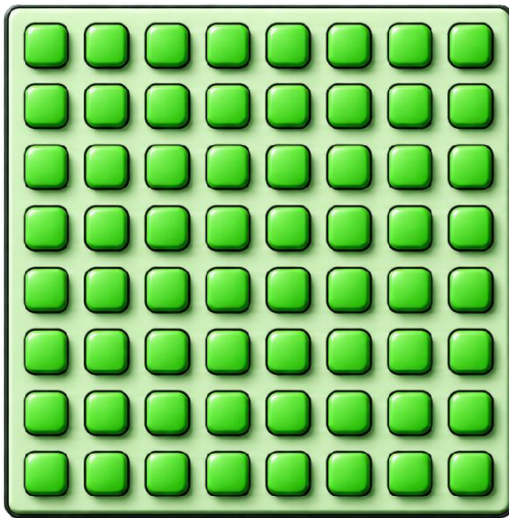
PATRONES DE GESTIÓN DE DATOS PARALELOS: PIPELINE

- ❑ Pipeline conecta las tareas en un modo productor-consumidor
- ❑ Un pipeline lineal es la idea básica del patrón, pero también es posible un pipeline en un DAG
- ❑ Los pipeline son más útiles cuando se utilizan con otros patrones, ya que pueden multiplicar el paralelismo disponible.



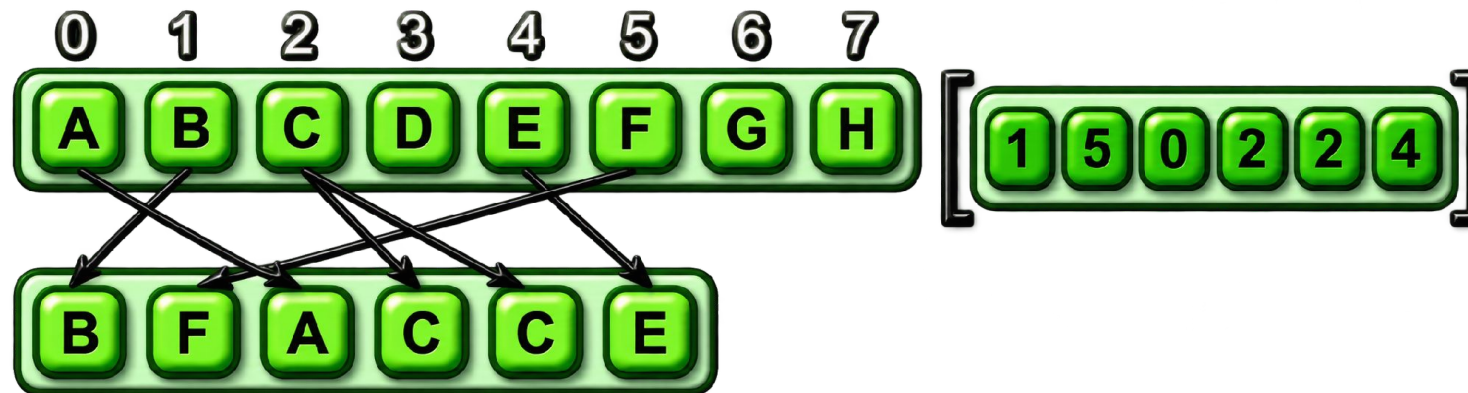
PATRONES DE GESTIÓN DE DATOS PARALELOS: GEOMETRIC DECOMPOSITION

- ❑ Descomposición geométrica: organiza los datos en subcolecciones
- ❑ Son posibles las descomposiciones superpuestas y no superpuestas
- ❑ Este patrón no necesariamente mueve datos, solo nos da otra vista de ellos



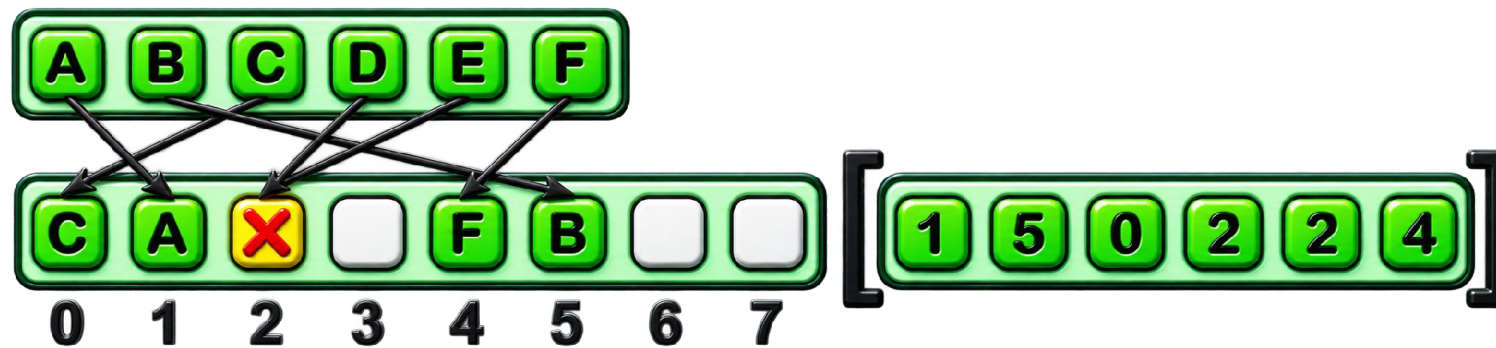
PATRONES DE GESTIÓN DE DATOS PARALELOS: GATHER

- ❑ Gather lee una colección de datos dada [a partir de] una colección de índices
- ❑ Piense en una combinación de map y lecturas seriadas aleatorias [random serial read]
- ❑ La colección de salida comparte el mismo tipo que la colección de entrada, pero comparte la misma forma que la colección de índices



PATRONES DE GESTIÓN DE DATOS PARALELOS: SCATTER

- ❑ Scatter es el inverso de gather
- ❑ Se requiere un conjunto de entradas e índices, pero cada elemento de la entrada se escribe en la salida en el índice dado en lugar de leerse en la entrada en el índice dado.
- ❑ Condiciones de Carrera puede ocurrir cuando tenemos dos escrituras en la misma ubicación!



OTROS PATRONES PARALELOS

1. Superscalar Sequences: escribe una secuencia de tareas, ordenadas solo por dependencias
2. Futures: similar a fork-join, pero las tareas no necesitan estar jerarquizadas
3. Speculative Selection: versión generalizada de la selección en serie donde la condición y ambos resultados pueden ejecutarse en paralelo
4. Workpile: patron map general map donde cada instancia de función elemental puede generar más instancias, agregando a la "pila" de trabajo

OTROS PATRONES PARALELOS

5. Search: encuentra algunos datos en una colección que cumple con algunos criterios
6. Segmentation: operaciones en particiones subdivididas, no superpuestas y de tamaño no uniforme de colecciones 1D
7. Expand: una combinación de pack y map
8. Category Reduction: dada una colección de elementos, cada uno con una etiqueta, busque todos los elementos con la misma etiqueta y redúzcalos

SOPORTE DE MODELOS DE PROGRAMACIÓN PARA PATRONES

Tabla 3.1 Resumen del soporte de los modelos de programación para los patrones secuenciales discutidos en este libro. Note que algunos de los modelos de programación paralela que consideramos, en realidad, no soportan todos los patrones de programación secuencial comunes. En particular, observe que la recursión y la asignación de memoria están limitadas en algunos modelos.

Patrón secuencial	TBB	Cilk Plus	OpenMP	ArBB	OpenCL
Anidamiento (secuencial)	F	F	F	F	F
Secuencia	F	F	F	F	F
Selección	F	F	F	F	F
Iteración	F	F	F	F	F
Recursión	F	F	F		?
Lectura aleatoria	F	F	F	F	F
Escritura aleatoria	F	F	F		F
Asignación en la pila	F	F	F		?
Asignación en el montón (heap)	F	F	F		
Clausuras (funciones anónimas)				F	F
Objetos	F	F	F (con C++)	F	

SOPORTE DE MODELOS DE PROGRAMACIÓN PARA PATRÓN

Tabla 3.2 Resumen del soporte de los modelos de programación para los patrones discutidos en este libro. F: Soportado directamente, con una característica especial. I: Puede implementarse fácil y eficientemente usando otras características. P: Implementaciones de un patrón en términos de otros, listadas bajo el patrón que se está implementando. En blanco significa que el patrón particular no puede implementarse en ese modelo de programación (o que una implementación eficiente no puede implementarse fácilmente). Cuando en este libro existen ejemplos de un patrón particular con un modelo particular, se indican referencias de sección.

Patrón paralelo	TBB	Cilk Plus	OpenMP	ArBB	OpenCL
Anidamiento paralelo	F	F			
Map	F 4.2.3; 4.3.3; 11	F 4.2.4; 4.2.5; 4.3.4; 4.3.5; 11	F 4.2.6; 4.3.6	F 4.2.7; 4.2.8; 4.3.7	F 4.2.9; 4.3.8
Stencil	I 10	I 10	I	F 10	I
Bolsa de trabajo	F				I
Reducción	F 5.3.4; 11	F 5.3.5; 11	F 5.3.6	F 5.3.7	I
Scan	F 5.6.5; 14	I 5.6.3; P 8.11; 14	I 5.6.4; P 5.4.4	F 5.6.6	I
Fork-join	F 8.9.2; 13	F 8.7; 8.9.1; 13	I		
Recurrencia		P 8.12			
Secuencia superescalar					
Futuros					F
Selección especulativa					
Pack	I 14	I 14	I	F	I
Expand	I	I	I	I	I
Pipeline	F 12	I 12	I		
Descomposición geométrica	I 15	I 15	I	I	I
Búsqueda	I	I	I	I	I
Reducción por categorías	I	I	I	I	I
Gather	I	F	I	F	I
Dispersión atómica	F	I	I		I
Dispersión por permutación	F	F	F	F	F
Dispersión por mezcla	I	I	I	F	I
Dispersión por prioridad					

SOPORTE DE MODELOS DE PROGRAMACIÓN PARA PATRONES

Tabla 3.3 Patrones adicionales discutidos.

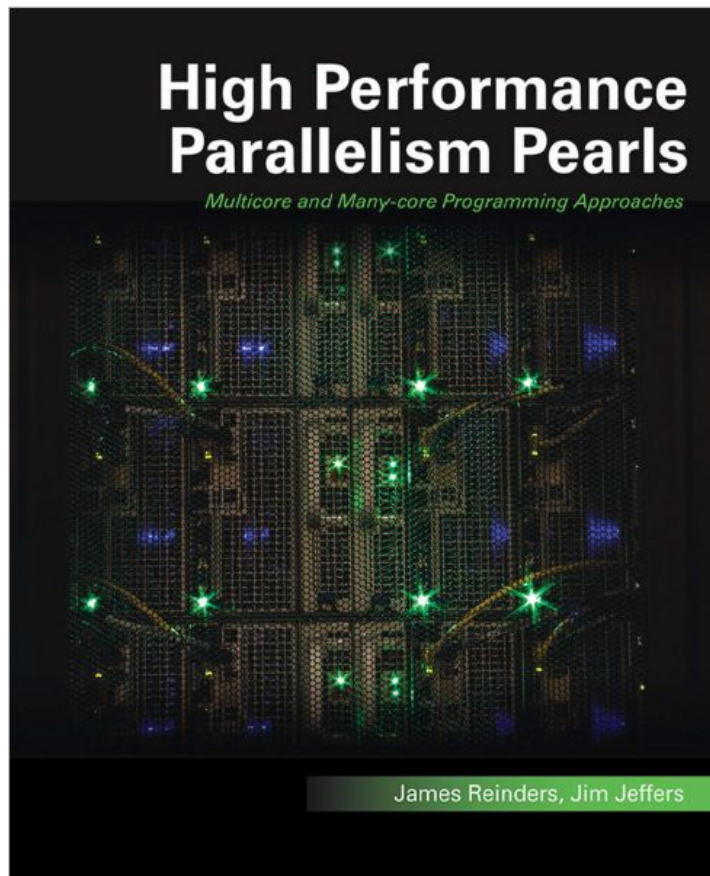
F: Compatible directamente, con una característica especial.

I: Se puede implementar fácil y eficientemente utilizando otras características.

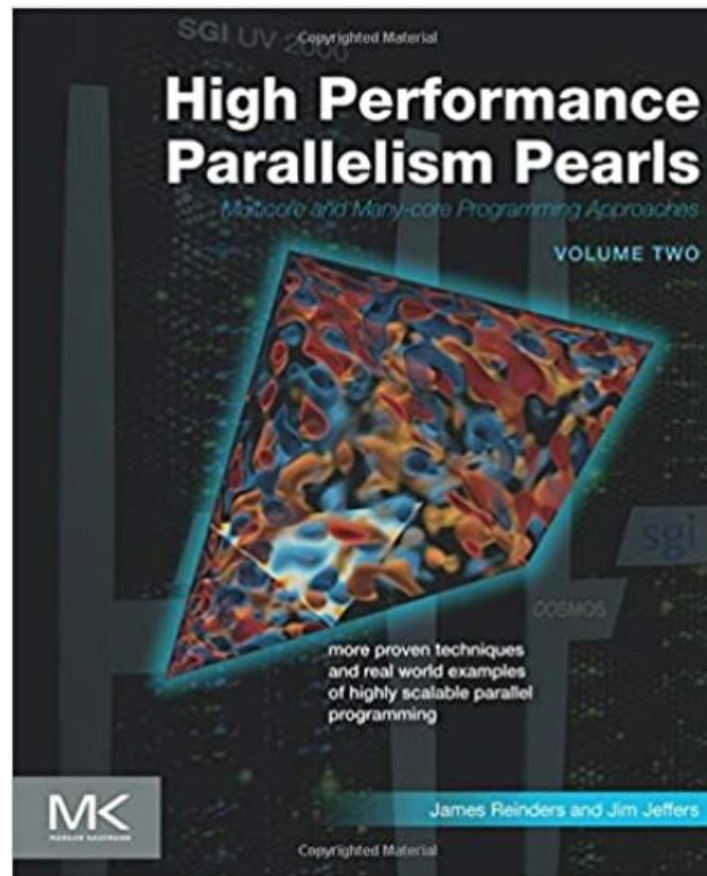
(celda en blanco): el patrón particular no se puede implementar en ese modelo de programación (o una implementación eficiente no puede realizarse fácilmente).

Patrón de paralelismo	TBB	Cilk Plus	OpenMP	ArBB	OpenCL
Secuencia superescalar	I	I	I		F
Futuros	I	I	I		I
Selección especulativa	I				
Workpile (bolsa de trabajo)	F	I	I		I
Expandir (Expand)	I	I	I	I	I
Search (búsqueda)	I	I	I	I	I
Reducción por categoría	I	I	I	I	I
Scatter atómico	F	I	I		I
Scatter por permutación	F	F	F	F	F
Scatter de fusión (merge)	I	I	I	F	I
Scatter por prioridad					

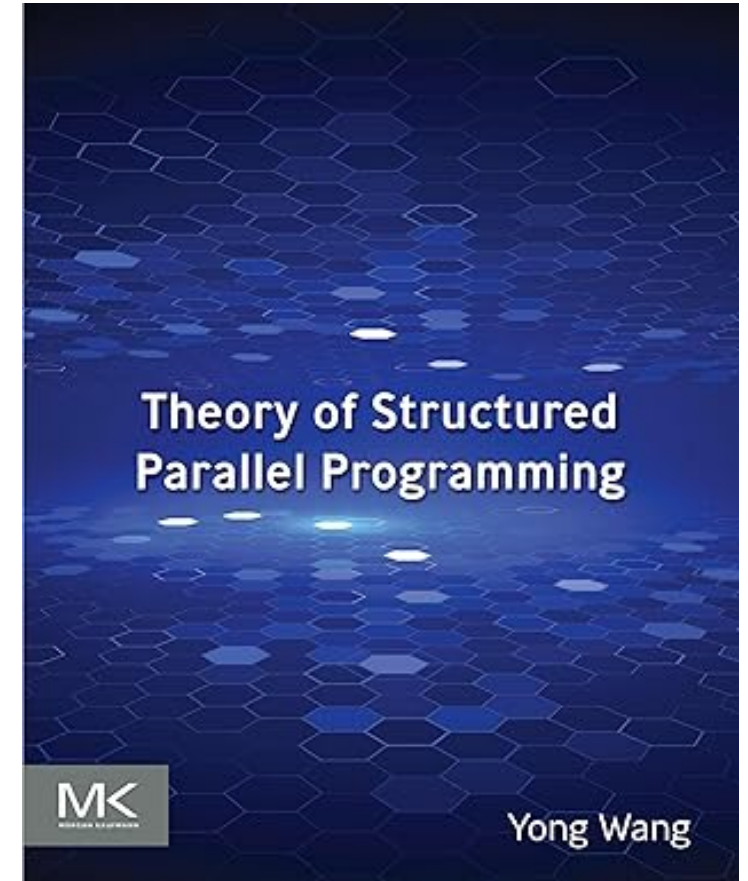
TRES LIBROS MÁS...



2014



2015



2024

PARA TERMINAR: COMPILANDO PARA PARALELISMO.

1. <http://www.tutorial-reports.com/computer-science/parallel-compiler/tutorial.php>
2. <https://cseweb.ucsd.edu/~berman/cse260/parcomp.html>
3. https://en.wikipedia.org/wiki/Automatic_parallelization
4. <https://rustc-dev-guide.rust-lang.org/parallel-rustc.html>
5. <https://gcc.gnu.org/wiki/ParallelGcc>
6. https://docs.nvidia.com/cuda/cuda-compiler-driver-nvcc/index.html#cuda-compilation-trajectory__cuda-compilation-from-cu-to-executable

REFERENCIAS

- McCool, M., Robison, A. D., & Reinders, J. (2012). *Structured parallel programming* (primera edición). Elsevier.

GRACIAS